Feedforward Networks in torch

2026-04-17

Introduction

`torch` for R is the native R port of PyTorch: an automatic-differentiation library with GPU support, a comprehensive layer library, and modern optimisers (Adam, AdamW). It enables building arbitrary neural-network architectures from scratch and training them with the same APIs Python users know. The companion package `luz` wraps `torch` with a scikit-learn-style `fit`/`predict` interface, abstracting away the manual training loop for routine cases.

Prerequisites

A working understanding of neural networks (forward propagation, backpropagation), gradient descent, the chain rule, and basic mini-batch training concepts.

Theory

A `torch` model is an `nn_module` — a composition of parameterised layers with a forward-pass method. Training iterates over epochs and mini-batches:

1. **Forward pass**: compute predictions and loss on a mini-batch.
2. **Backward pass**: compute gradients via reverse-mode automatic differentiation.
3. **Optimiser step**: update parameters using the gradient (Adam adjusts step sizes per parameter).

Modern training loops add learning-rate scheduling (cosine annealing, warmup), gradient clipping, weight decay, and early stopping. `luz` packages these patterns into named callbacks.

Assumptions

Sufficient data for the chosen model capacity (overparameterised models on tiny datasets memorise rather than generalise); features standardised before input; mini-batch size appropriate for available memory and the bias-variance trade-off.

R Implementation

```
library(torch)

set.seed(2026)
torch_manual_seed(2026)

# Simulate a simple regression task
n <- 1000
x <- torch_randn(n, 3)
y <- (x[, 1] * 2 + x[, 2]^2 - x[, 3] * 0.5)$unsqueeze(2)

net <- nn_module(
  initialize = function() {
    self$fc1 <- nn_linear(3, 16)
```

```
    self$fc2 <- nn_linear(16, 16)
    self$fc3 <- nn_linear(16, 1)
  },
  forward = function(x) {
    x %>% self$fc1() %>% nnf_relu() %>%
          self$fc2() %>% nnf_relu() %>%
          self$fc3()
  }
)

model <- net()
opt   <- optim_adam(model$parameters, lr = 1e-2)

for (epoch in 1:50) {
  opt$zero_grad()
  pred <- model(x)
  loss <- nnf_mse_loss(pred, y)
  loss$backward()
  opt$step()
  if (epoch %% 10 == 0)
    cat(sprintf("epoch %d  loss %.4f\n", epoch, loss$item()))
}
```

Output & Results

Per-epoch MSE loss should decrease monotonically as the network learns the non-linear regression. The 3-layer MLP with ReLU activations rapidly fits the simulated quadratic-in-x_2 structure that linear regression cannot capture.

Interpretation

A reporting sentence: "A 3-layer feedforward network (16-16-1 hidden units, ReLU activations, Adam optimiser, learning rate 0.01) trained for 50 epochs reduced MSE from 3.1 to 0.04 on the simulated regression with quadratic interactions; predictions closely tracked the noisy targets, with residuals consistent with the irreducible simulation noise level." Always state the architecture, optimiser, learning rate, and training epochs.

Practical Tips

- Use `luz::fit()` for a scikit-learn-style interface that handles the training loop, dataloaders, and standard callbacks (early stopping, model checkpointing, learning-rate scheduling).
- Move tensors and models to GPU with `$to(device = "cuda")`; check `cuda_is_available()` first to handle CPU-only systems gracefully.
- Mini-batching via `dataloader()` is essential beyond toy problems; full-batch gradient descent is computationally inefficient and produces noisier optimisation trajectories than expected.
- Monitor training and validation loss separately; early-stop when validation loss plateaus or starts to rise to prevent over-fitting.
- `torchvision` and `torchaudio` add pre-trained models for transfer learning; these dramatically reduce training time and data requirements for common tasks (image classification, audio recognition).
- For tabular data, gradient-boosted trees (XGBoost, LightGBM) usually outperform feedforward networks; deep learning shines on raw modalities (images, text, audio) where structure is encoded in the data layout.

R Packages Used

`torch` for the canonical PyTorch-like API in R; `luz` for sklearn-style `fit`/`predict`; `torchvision`, `torchaudio`, `tabnet` for domain-specific models; `torcheval` for evaluation metrics; `coro` for Python-style generators (used internally by `dataloader`).

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis